Lab 2: Characterising the Physical Channel (Computer Networks: Physical Layer / Transmission Media)

In this lab, we compute the physical limits of network cabling considering three grades of copper twisted-pair cable (Cat5e, Cat6, and Cat6a) and for each one we calculate how fast it can carry data, how far a signal travels before it needs a repeater, and how often bits arrive wrong. We also compare copper's reach against optical fibre.

Specifically, you will:

- turn a cable's bandwidth and SNR into a maximum bit rate, using Nyquist and Shannon;
- work out copper's maximum distance from its attenuation, compare it against fibre, and derive the 100 m Ethernet limit;
- find the range of a wireless link from free-space path loss;
- Express a channel's noise level as a bit-error rate.

Setup

```
python3 -m pip install numpy matplotlib      # once
python3 lab2_template.py
```

Using the lab2_template.py (where core formulas left blank) your job is to fill them in. The three cases and the fixed settings, Parts A and B compare three cable grades.

Case	Cable	Bandwidth
1	Cat5e	100 MHz
2	Cat6	250 MHz
3	Cat6a	500 MHz

Parameters: channel SNR = 25 dB, transmitter power = 0 dBm, receiver sensitivity = −40 dBm (a 40 dB link budget), long-haul copper α = 8 dB/km, fibre α = 0.3 dB/km. Part C compares three wireless bands: **900 MHz, 2.4 GHz, 5 GHz**.

Part A — Bandwidth → bit rate

For each of the three cable grades (Cat5e, Cat6, Cat6a), determine the maximum bit rate it can carry under two conditions. First compute the noiseless limit using Nyquist's formula, then compute the noise-limited capacity using Shannon's formula.

Plot how that capacity varies with SNR, and identify which of the two computed values represents the true achievable limit.

1. **Nyquist (noiseless).** A channel of bandwidth B allows at most 2B **symbols per second** (the baud rate); each symbol carries log (M) bits, where M is the number of signal levels. So bit rate = baud × bits-per-symbol = 2·B· log (M). Rearrange the equation to solve for the number of levels: M = 2^(rate / 2B). Set the target rate to 1 Gb/s and compute M for each of the three cables. Verify your results against the expected values: 32 for Cat5e, 4 for Cat6, and 2 for Cat6a. State the relationship these values demonstrate: the cable with the greatest bandwidth requires the fewest signal levels to achieve the same bit rate.
2. **Shannon (noisy).** C = B·log (1 + S/N), with S/N = 10^(25/10). Compute the capacity ceiling for each cable.
3. **Plot** capacity vs SNR (0–40 dB) with one curve per cable, and mark the 25 dB line.
4. **Three ceilings.** You now have two numbers per cable — the Nyquist figure (step 1) and the Shannon figure (step 2). Which is the *fantasy* ceiling and which is the *real* one, and why? Where would you expect actual hardware to land relative to the two?

Part B — Attenuation → distance

For a copper link and a fibre link, calculate how far the signal travels before it becomes too weak to detect. Then repeat the calculation for a Cat5e cable operating at 100 MHz to derive the 100 m Ethernet distance limit.

Background: as a signal travels along a medium, its power decreases with distance. The loss is measured in decibels per kilometre and is written α (attenuation). The received power after a distance d is:

$P(d) = P_{tx} - \alpha \cdot d$

The receiver stops working once the received power falls below a fixed threshold called the receiver sensitivity, P_sens. Setting P(d) = P_sens and solving for d gives the maximum distance:

$d_{max} = (P_{tx} - P_{sens}) / \alpha$

The quantity P_tx − P_sens is the link budget: the total signal loss the link can tolerate before failing.

a. State the parameters. Transmit power P_tx = 0 dBm, receiver sensitivity P_sens = −40 dBm. Confirm that the link budget is therefore 0 − (−40) = 40 dB.
b. Compute the copper distance. Using α = 8 dB/km, calculate d_max. The result should be approximately 5 km, which matches the value quoted in the lecture.
c. Compute the fibre distance. Using α = 0.3 dB/km, calculate d_max. Report how many times further fibre reaches than copper.

d. Explain the fibre result. State why a real fibre link can reach even further than this calculation suggests. (Consider the use of optical amplifiers and repeaters, and fibre's lower receiver sensitivity threshold.)
e. Derive the Ethernet limit. A Cat5e cable operating at 100 MHz has a much higher attenuation, α = 220 dB/km, because attenuation increases with frequency. Using a link budget of 22 dB, calculate d_max. Confirm that the result is 100 m. This is the physical origin of the 100 m limit on copper Ethernet.

Plot the result. Plot received power against distance for both copper and fibre on the same axes, and draw a horizontal line at the receiver sensitivity of −40 dBm to mark where each link fails.

Part C: Wireless

For a wireless link, calculate how much signal power is lost over distance at three different frequencies, and determine why a higher frequency produces a shorter range.

Background: a wireless signal is not confined to a cable. It spreads outward through free space, and its power decreases with both distance and frequency. This loss is called free-space path loss (FSPL) and is given by the Friis equation:

`FSPL(dB) = 20·log (d) + 20·log (f) + 20·log (4π / c)`

where `d` is the distance between transmitter and receiver in metres, `f` is the signal frequency in hertz, and `c` is the speed of light, `3 × 10 ` m/s. The transmitter emits at a fixed power of `P_tx = 0 dBm`, and the received power is `P_rx = P_tx − FSPL`. The frequency `f` appears inside the loss expression, and the loss increases as `f` increases, so a higher-frequency signal is attenuated more strongly over the same distance.

Your task:

1. Compute the received power at a distance of 5 km for three frequencies: 900 MHz, 2.4 GHz, and 5 GHz.

2. Report the difference in received power between 900 MHz and 5 GHz. The difference should be approximately 15 dB. Identify the term in the Friis equation responsible for this gap.

3. Plot received power against distance from 0 to 20 km, drawing one curve for each of the three frequencies.

Part D: Bit error rate

Calculate how often bits are received incorrectly as the signal-to-noise ratio changes, then summarise the overall quality of the channel in a single line.

Background: even when a link is operational, noise occasionally causes a received bit to be read as the wrong value. The fraction of bits received in error is the bit error rate (BER).

Two distinct signal-to-noise quantities appear in this lab, and they must not be confused:

- The channel SNR used in Shannon's formula (25 dB) is the total signal power divided by the total noise power. It answers a capacity question: how many signal levels the channel can support.

- Eb/N0 is the signal-to-noise ratio per bit of information. `Eb` is the energy carried by one bit, and `N0` is the noise power per hertz of bandwidth. It answers a reliability question: whether a given bit is received correctly.

Whether a bit is received in error depends on Eb/N0, not on the channel SNR, so BER is expressed as a function of Eb/N0. For a BPSK signal in noise:

`BER = 0.5 · erfc(√(Eb/N0_linear))`

where `Eb/N0_linear` is the ratio expressed as a linear value rather than in decibels, and `erfc` is the complementary error function, available as `math.erfc`.

1. Compute the BER across a range of Eb/N0 values from 0 to 12 dB.
2. Plot the BER against Eb/N0, using a logarithmic scale on the vertical axis.
3. Identify the value of Eb/N0 at which the BER reaches 10^{-3}. The result should be approximately 6.8 dB.
4. Write a one-line summary of the channel, reporting its maximum span, its capacity, its operating Eb/N0, and its BER. Use an operating point of Eb/N0 = 7 dB, which corresponds to a BER of approximately 8×10 .

To be submitted:

- Three plots: capacity vs SNR for the three cables (A), power vs distance for copper/fibre (B), BER vs Eb/N0 (D). Wireless plot (C) optional but encouraged.
- Your derived numbers: levels for 1 Gb/s per cable, Shannon capacities, copper/fibre spans, the 100 m derivation, and your one-line channel summary.

Appendix: Starter template (lab2_template.py)

```
import math
import numpy as np
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C_LIGHT = 3e8
TX_DBM, RX_SENS_DBM = 0.0, -40.0
SNR_DB   = 25
CU_ALPHA, FI_ALPHA = 8.0, 0.3
CAT5E_ALPHA_100MHZ, LAN_BUDGET_DB = 220.0, 22.0

CABLES = [("Cat5e", 100e6), ("Cat6", 250e6), ("Cat6a", 500e6)]
BANDS  = [900e6, 2400e6, 5000e6]

# ---- Fill these in ----------------------------------------------------
def snr_db_to_linear(snr_db):
    # TODO: convert dB to a linear ratio
    pass

def levels_for_rate(B, target_rate):
    # TODO (Part A): invert Nyquist  C = 2B log2(M)  ->  M
    pass

def shannon_capacity_db(B, snr_db):
    # TODO (Part A): C = B log2(1 + S/N)
    pass

def max_span_km(tx_dbm, rx_sens_dbm, alpha_db_km):
    # TODO (Part B): distance until power hits receiver sensitivity
    pass

def fspl_db(d_m, f_hz):
    # TODO (Part C): free-space path loss (Friis)
    pass

def ber_bpsk(ebn0_db):
    # TODO (Part D): 0.5 * erfc(sqrt(Eb/N0_linear))
    pass
# -----------------------------------------------------------------------

def run():
    print("Part A  Bandwidth -> bit rate  (SNR = %d dB)" % SNR_DB)
    for name, B in CABLES:
        print(" ", name, levels_for_rate(B, 1e9), shannon_capacity_db(B, SNR_DB))
    print("Part B  copper/fibre span, Cat5e@100MHz reach:",
          max_span_km(TX_DBM, RX_SENS_DBM, CU_ALPHA),
          max_span_km(TX_DBM, RX_SENS_DBM, FI_ALPHA),
          LAN_BUDGET_DB / CAT5E_ALPHA_100MHZ * 1000)
    print("Part C  received power at 5 km per band:")
    for f in BANDS:
        print("  ", f/1e6, "MHz:", TX_DBM - fspl_db(5000, f))
```

```
    # Part A plot: one capacity curve per cable
    s = np.linspace(0, 40, 200)
    plt.figure()
    for name, B in CABLES:
        plt.plot(s, [shannon_capacity_db(B, x)/1e6 for x in s], label=name)
    plt.axvline(SNR_DB, ls='--', color='grey')
    plt.legend(); plt.xlabel("SNR (dB)"); plt.ylabel("Capacity (Mb/s)"); plt.title("A")
    plt.savefig("partA.png")

    # Part B plot
    d = np.linspace(0.01, 60, 400)
    plt.figure()
    plt.plot(d, TX_DBM - CU_ALPHA*d, label="copper")
    plt.plot(d, TX_DBM - FI_ALPHA*d, label="fibre")
    plt.axhline(RX_SENS_DBM, ls='--', color='grey')
    plt.legend(); plt.xlabel("distance (km)"); plt.ylabel("dBm"); plt.title("B")
    plt.savefig("partB.png")

    # Part C plot: one curve per band
    dm = np.linspace(10, 20000, 400)
    plt.figure()
    for f in BANDS:
        plt.plot(dm/1000, [TX_DBM - fspl_db(x, f) for x in dm], label=f"{f/1e6:.0f} MHz")
    plt.legend(); plt.xlabel("distance (km)"); plt.ylabel("dBm"); plt.title("C")
    plt.savefig("partC.png")

    # Part D plot
    e = np.linspace(0, 12, 400)
    plt.figure(); plt.semilogy(e, [ber_bpsk(x) for x in e])
    plt.axhline(1e-3, ls='--', color='crimson')
    plt.xlabel("Eb/N0 (dB)"); plt.ylabel("BER"); plt.title("D")
    plt.savefig("partD.png")

if __name__ == "__main__":
    run()
```